



N-gram Language Models

*Maryam Abdolali
KNTU, Fall 2025*

Topics

P1

N-GRAM INTUITION

P2

EVALUATION & PERPLEXITY

P3

SAMPLING & GENERATION

P4

SMOOTHING, INTERPOLATION, BACK-OFF

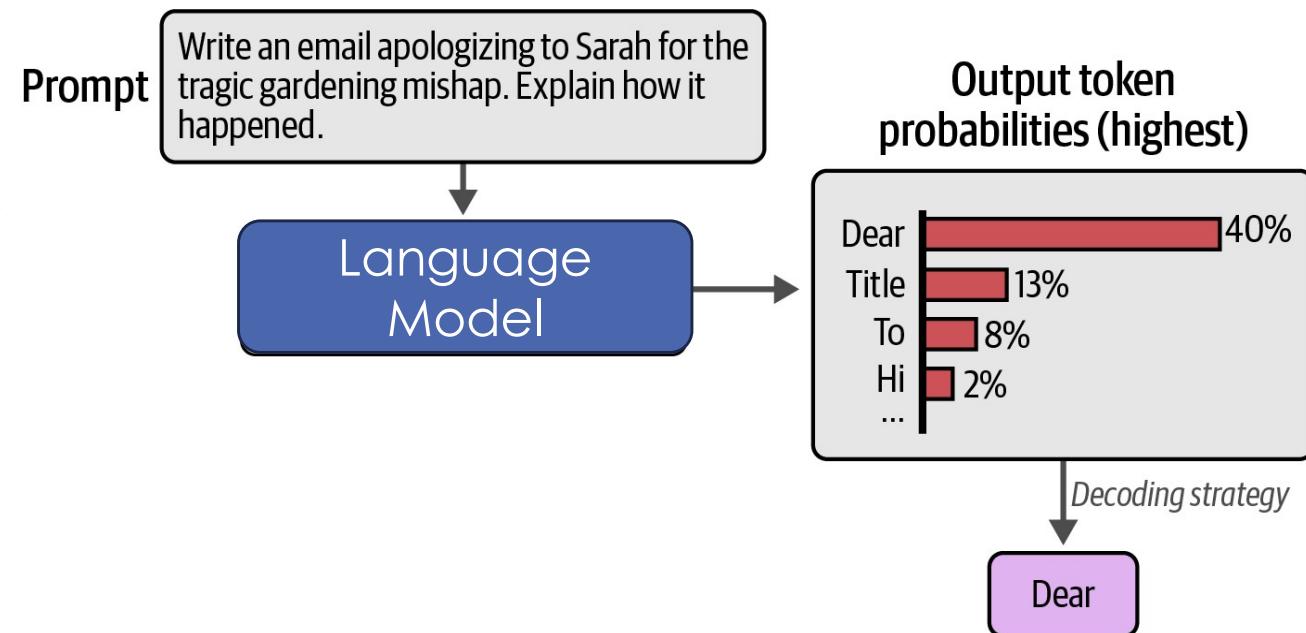
Language Model

A language model is a machine learning model that predicts upcoming words.

- More formally:
 - An LM assigns a probability to each potential next word
= gives a probability distribution over possible next words
 - An LM assigns a probability to a whole sentence

Two paradigms

- N-gram language models (this lecture)
- Large language models (LLMs, future lectures)



Predicting words... is it possible?

- “The water of Walden Pond is beautifully ...”

What word is likely to follow...?

blue

green

clear

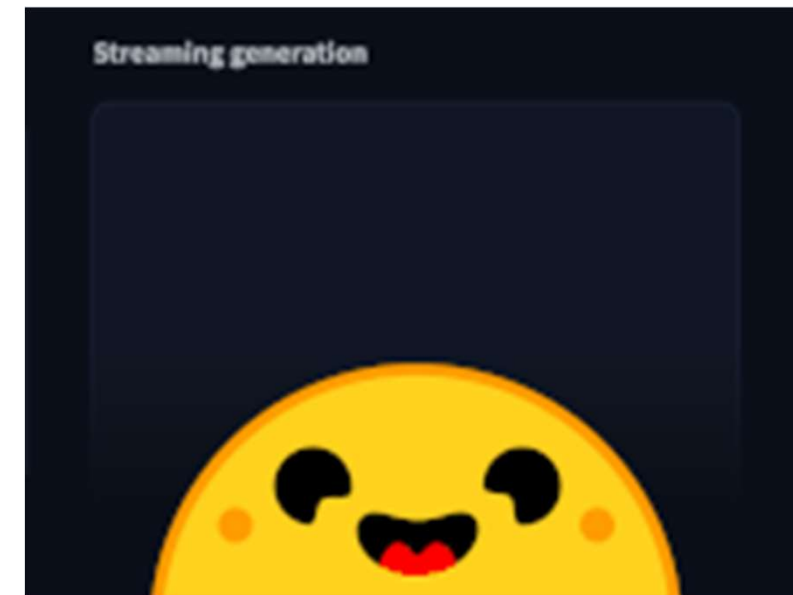
*refrigerator

*that

It's how **large language models (LLMs)** work!

LLMs are **trained** to predict words

- Left-to-right (autoregressive) LMs learn to predict next word



Language Modeling (LM) more formally

- **Goal:** compute the probability of a sentence or sequence of words W :

$$P(W) = P(w_1, w_2, w_3, w_4, w_5 \dots w_n)$$

- **Related task:** probability of an upcoming word:

$$P(w_5 | w_1, w_2, w_3, w_4) \text{ or } P(w_n | w_1, w_2 \dots w_{n-1})$$

- An LM computes either of these:

$$P(W) \quad \text{or} \quad P(w_n | w_1, w_2 \dots w_{n-1})$$

- How to estimate these probabilities?

✓ Could we just count and divide?

$P(\text{blue} | \text{The water of Walden Pond is so beautifully})$

$$\frac{C(\text{The water of Walden Pond is so beautifully blue})}{C(\text{The water of Walden Pond is so beautifully})}$$

✓ **No!** Too many possible sentences!

✓ We'll **never see enough data** for estimating these

How to compute $P(W)$ or $P(w_n|w_1, \dots, w_{n-1})$ with an N -gram Language Model

- First the **joint probability** $P(W)$:

$P(\text{The, water, of, Walden, Pond})$

- Intuition:** let's rely on the Chain Rule of Probability

- Chain rule reminder:

- $P(A,B,C,D) = P(A) P(B|A) P(C|A,B) P(D|A,B,C)$

$$\begin{aligned} P(w_{1:n}) &= P(w_1)P(w_2|w_1)P(w_3|w_{1:2}) \dots P(w_n|w_{1:n-1}) \\ &= \prod_{k=1}^n P(w_k|w_{1:k-1}) \end{aligned}$$

SO $P(\text{"The water of Walden Pond"}) =$

$$\begin{aligned} &P(\text{The}) \times P(\text{water} | \text{The}) \times P(\text{of} | \text{The water}) \\ &\times P(\text{Walden} | \text{The water of}) \times P(\text{Pond} | \text{The water of Walden}) \end{aligned}$$

Markov Assumption

- ✓ To approximate the **probability of a word** given all the previous words/history $P(w_n|w_{1:n-1})$ by using only the conditional probability given the **preceding word** $P(w_n|w_{n-1})$.
- ✓ Markov models are the class of probabilistic models that assume we can **predict the probability of some future unit without looking too far into the past**.

N-gram Models

In general, we had: $P(w_{1:n}) = \prod_{k=1}^n P(w_k | w_{1:k-1})$

- *Bigram Models:*

$$P(w_{1:n}) \approx \prod_{k=1}^n P(w_k | w_{k-1})$$

$P(\text{blue} | \text{The water of Walden Pond is so beautifully})$

$P(\text{blue} | \text{beautifully})$



- *General N-gram model:*

$$P(w_n | w_{1:n-1}) \approx P(w_n | w_{n-N+1:n-1})$$

Generating sentences from Shakespeare bigram models

4 different N-gram models

1
gram

–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have

–Hill he late speaks; or! a more to leg less first you enter

2
gram

–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.

–What means, sir. I confess she? then all sorts, he is trim, captain.

3
gram

–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.

–This shall forbid it should be branded, if renown made it empty.

4
gram

–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;

–It cannot be but so.

we give the model
the ability to see
more context, the
sentences get
more and more
like English

More on Generating later...

Problems with N-gram models

1. N-grams can't handle **long-distance dependencies**:

“**The soups** that I made from that new cookbook I bought yesterday **were** amazingly delicious.”

the correct verb **were** depends on **soups** many words back. A **3-gram or 5-gram can't “see” that far**; even a 10-gram is brittle because the exact intervening words vary. So it falls back on local surface stats and often prefers the wrong agreement (e.g., *was*).

2. N-grams don't do well at **modeling new sequences** (even if they have similar meanings to sequences they've seen)

N-grams memorize counts of seen N-length strings. Any new phrase (even with familiar meaning) gets near-zero counts. Can't infer that “cookbook I bought yesterday” $\approx$ “recipe book I purchased yesterday.”

The solution: Large language models

- can handle much longer contexts
- because they use neural **embedding spaces**, can model meaning better

Estimating bigram probabilities

- The Maximum Likelihood Estimate

$$P(w_n | w_{n-1}) = \frac{C(w_{n-1}w_n)}{\sum_w C(w_{n-1}w)}$$

$$P(w_n | w_{n-1}) = \frac{C(w_{n-1}w_n)}{C(w_{n-1})}$$

- Example:

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

$$P(\text{I} | \text{<s>}) = \frac{2}{3} = .67$$

$$P(\text{Sam} | \text{<s>}) = \frac{1}{3} = .33$$

$$P(\text{am} | \text{I}) = \frac{2}{3} = .67$$

$$P(\text{</s>} | \text{Sam}) = \frac{1}{2} = 0.5$$

$$P(\text{Sam} | \text{am}) = \frac{1}{2} = .5$$

$$P(\text{do} | \text{I}) = \frac{1}{3} = .33$$

More examples:

Berkeley Restaurant Project sentences

- ✓ Contains sample user queries (text-normalized, by lower casing and with punctuation striped)
- ✓ Samples of 9332 sentences is on the website:
 - can you tell me about any good cantonese restaurants close by
 - tell me about chez panisse
 - i'm looking for a good place to eat breakfast
 - when is caffe venezia open during the day

Raw bigram counts

- Bigram counts for **eight of the words** (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences. Zero counts are in gray.
- Each cell shows the **count of the column label word following the row label word**. Thus the cell in row “i” and column “want” means that *want* followed *i* 827 times in the corpus

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Raw bigram probabilities

- Normalize by unigrams:

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

- Result:

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

$$\begin{aligned} P(<s> I want english food </s>) = & \\ P(I | <s>) & \\ \times P(want | I) & \\ \times P(english | want) & \\ \times P(food | english) & \\ \times P(</s> | food) & \\ = .000031 & \end{aligned}$$

What kinds of knowledge do N-grams represent?

- $P(\text{to}|\text{want}) = .66$
- $P(\text{want} | \text{spend}) = 0$
- $P(\text{of} | \text{to}) = 0$

- $P(\text{dinner}|\text{lunch or}) = .83$
- $P(\text{dinner}|\text{for}) \sim P(\text{lunch}|\text{for})$

- $P(\text{chinese}|\text{want}) > P(\text{english}|\text{want})$

Knowledge of grammar

("want" is followed by infinitive "to")

("of" is not a verb)

Knowledge of meaning

The words "dinner" or "lunch" are semantically related.

Knowledge about the world

Chinese food is very popular

Dealing with scale in large n-grams

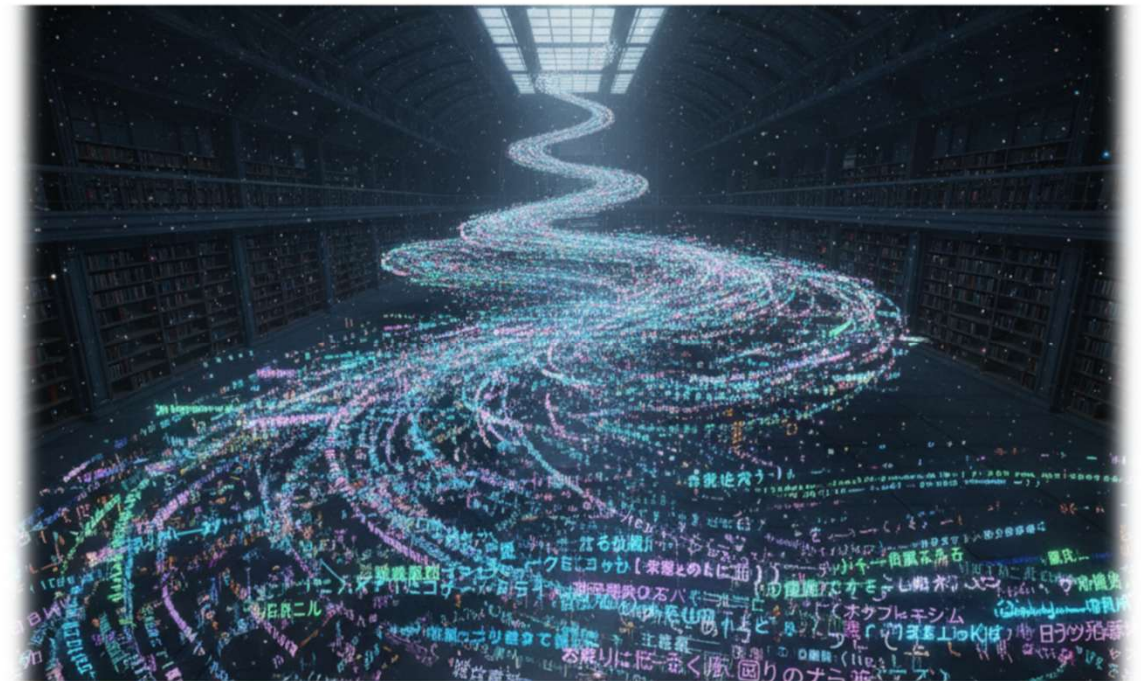
- LM probabilities are stored and computed in log format, i.e. **log probabilities**
- This avoids underflow from multiplying many small numbers

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

- If we need probabilities, we can do one exp at the end

$$p_1 \times p_2 \times p_3 \times p_4 = \exp(\log p_1 + \log p_2 + \log p_3 + \log p_4)$$

LONG LONG TEXTS



How to evaluate N-gram models

- "Extrinsic Evaluation"

To compare models A and B

1. Put each model in a **real task**
 - Machine Translation, speech recognition, etc.
2. Run the task, **get a score for A and for B**
 - How many words translated correctly
 - How many words transcribed correctly
3. **Compare accuracy** for A and B

- Extrinsic evaluation **not always possible**

- **Expensive, time-consuming**
- ✗ • we might want to see how well our n-gram language model works more generally, and not just in a particular application

- **Intrinsic evaluation: perplexity**

- Directly measures language model **performance at predicting words**.
- **Doesn't** necessarily correspond with **real application performance**
- But gives us a **single general metric** for language models
- **Useful for LLMs** as well as n-grams

Training sets and test sets

- ✓ We train parameters of our model on a **training set**.
- ✓ We test the model's performance on data we haven't seen.
 - A **test set** is an unseen dataset; different from training set.
 - Intuition: we want to measure generalization to unseen data
 - An **evaluation metric** (like **perplexity**) tells us how well our model does on the test set.

- ✓ Test set and the training set **must be non-overlapping**.
- ✓ If we have a test sentence in the training set, the n-gram will learn which words follow which words in that sentence, and then when we see it in the test set, we will assign it a very high probability, and that will falsely make the language model seem more accurate than it is.
- ✓ **Training on the test set is always a bad idea**

- If we're building an **LM for a specific task**
 - The **test set** should **reflect the task language** we want to use the model for
- If we're building a **general-purpose model**
 - We'll need **lots of different kinds of training data**
 - We don't want the **training set or the test set** to be just from one domain or author or language.

Intuition of perplexity as evaluation metric: How good is our language model?

Intuition: A good LM prefers "real" sentences

- Assign **higher probability** to “real” or “frequently observed” sentences
- Assigns **lower probability** to “word salad” or “rarely observed” sentences

The Shannon Game: **How well can we predict the next word?**

- Once upon a _____
- That is a picture of a _____
- For breakfast I ate my usual _____

Unigrams are terrible at this game (Why?)

time 0.9
dream 0.03
midnight 0.02
...
and 1e-100

A good LM is one that assigns a higher probability to the next word that actually occurs

Intuition of perplexity:

The best language model is one that best predicts the entire unseen test set

- We said: a good LM is one that assigns a higher probability to the next word that actually occurs.
- Let's generalize to all the words!
 - The best LM assigns high probability to the entire test set.
- When comparing two LMs, A and B
 - We compute $P_A(\text{test set})$ and $P_B(\text{test set})$
 - The better LM will give a higher probability to (=be less surprised by) the test set than the other LM.
- Probability depends on size of test set
 - Probability gets smaller the longer the text
 - Better: a metric that is *per-word*, normalized by length
- **Perplexity** is the inverse probability of the test set, normalized by the number of words

$$\begin{aligned} PP(W) &= P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} \\ &= \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}} \end{aligned}$$

Intuition of perplexity: The inverse

Perplexity is the **inverse** probability of the test set, normalized by the number of words $\frac{1}{N}$

$$PP(W) = P(w_1 w_2 \dots w_N)^{-\frac{1}{N}}$$

$$= \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}}$$

(The inverse comes from the original definition of perplexity from cross-entropy rate in information theory)

Probability range is $[0,1]$, perplexity range is $[1,\infty]$

Minimizing perplexity is the same as maximizing probability

Chain rule:

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_1 \dots w_{i-1})}}$$

Bigrams:

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_{i-1})}}$$

Intuition of perplexity:

Weighted average branching factor

Perplexity is also the **weighted average branching factor** of a language.

Branching factor: number of possible next words that can follow any word

Example: Deterministic language $L = \{\text{red}, \text{blue}, \text{green}\}$

Branching factor = 3 (any word can be followed by red, blue, green)

Now assume LM A where each word follows any other word with equal probability $\frac{1}{3}$

Given a test set $T = \text{"red red red red blue"}$

$$\text{Perplexity}_A(T) = P_A(\text{red red red red blue})^{-1/5} = ((\frac{1}{3})^5)^{-1/5} = (\frac{1}{3})^{-1} = 3$$

- But now suppose red was very likely in training set, such that for LM B:
 - $P(\text{red}) = .8$ $p(\text{green}) = .1$ $p(\text{blue}) = .1$
- We would expect the probability to be higher, and hence the perplexity to be smaller.

$$\begin{aligned}\text{Perplexity}_B(T) &= P_B(\text{red red red red blue})^{-1/5} \\ &= (.8 * .8 * .8 * .8 * .1)^{-1/5} = .04096^{-1/5} = .527^{-1} = 1.89\end{aligned}$$

Holding test set constant: Lower perplexity = better language model

- Training 38 million words, test 1.5 million words, WSJ

N-gram Order	Unigram	Bigram	Trigram
Perplexity	962	170	109

- ✓ So if we **hold the test set constant**, the better, more predictive the language model, the **lower the perplexity** will be.
- ✓ Here we can see this with some text from the **US Wall Street Journal newspaper text corpus**.
- ✓ A unigram LM has a perplexity of 962, a bigram of 170, and a trigram of 109.
- ✓ The **better the model, the higher the probability** it assigns to the test set, and since perplexity goes inversely with probability, **lower the perplexity**.

Example: Computing perplexity from sentence probabilities

- Suppose we have *trained* a small *language model* over an English corpus from a small subset of six words:

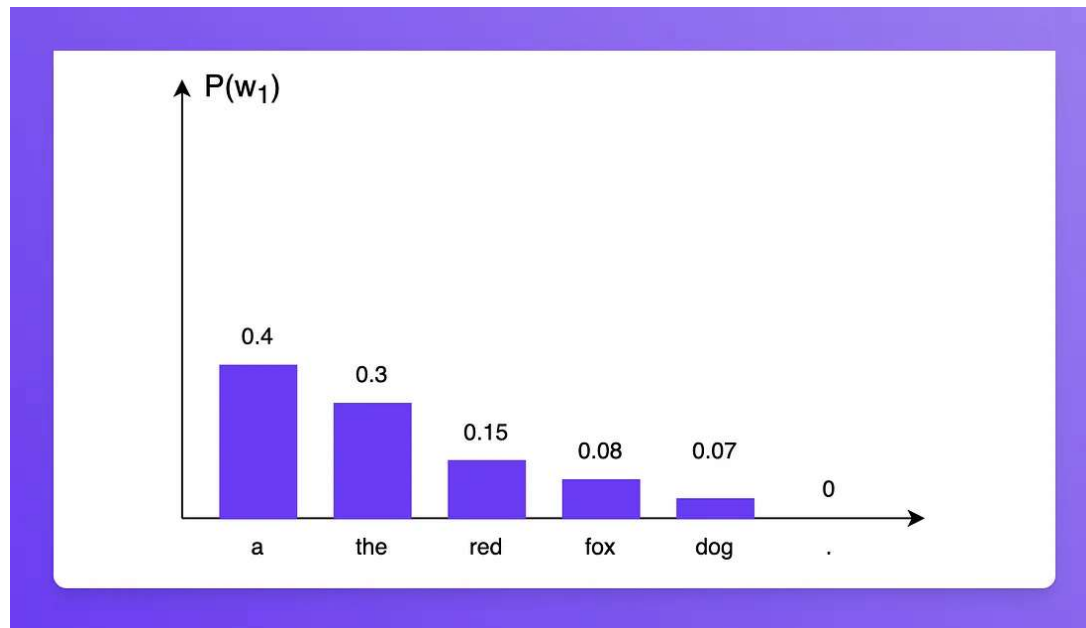
“a”, “the”, “red”, “fox”, “dog”, and “.”.

- Let's compute the *probability* of the sentence *“a red fox.”*.

$P(\text{“a red fox.”}) =$

$P(\text{“a”}) * P(\text{“red”} \mid \text{“a”}) * P(\text{“fox”} \mid \text{“a red”}) * P(\text{“.”} \mid \text{“a red fox”})$

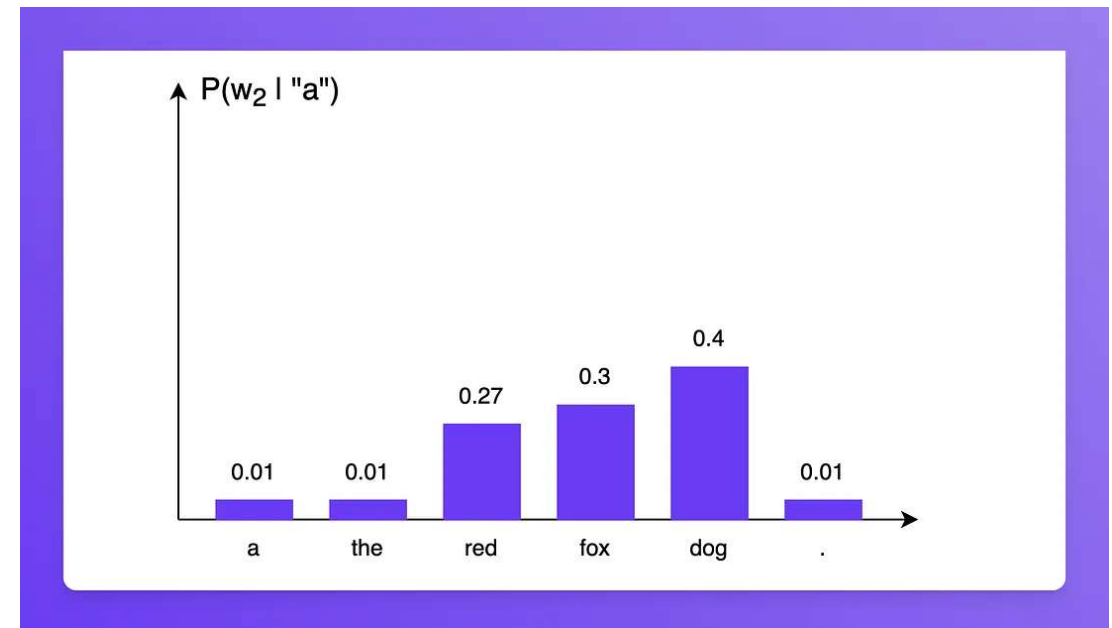
Suppose these are the probabilities assigned by our language model to a generic **first word** in a sentence:



As can be seen from the chart, the probability of “a” as the first word of a sentence is:

$$P(\text{“a”}) = 0.4$$

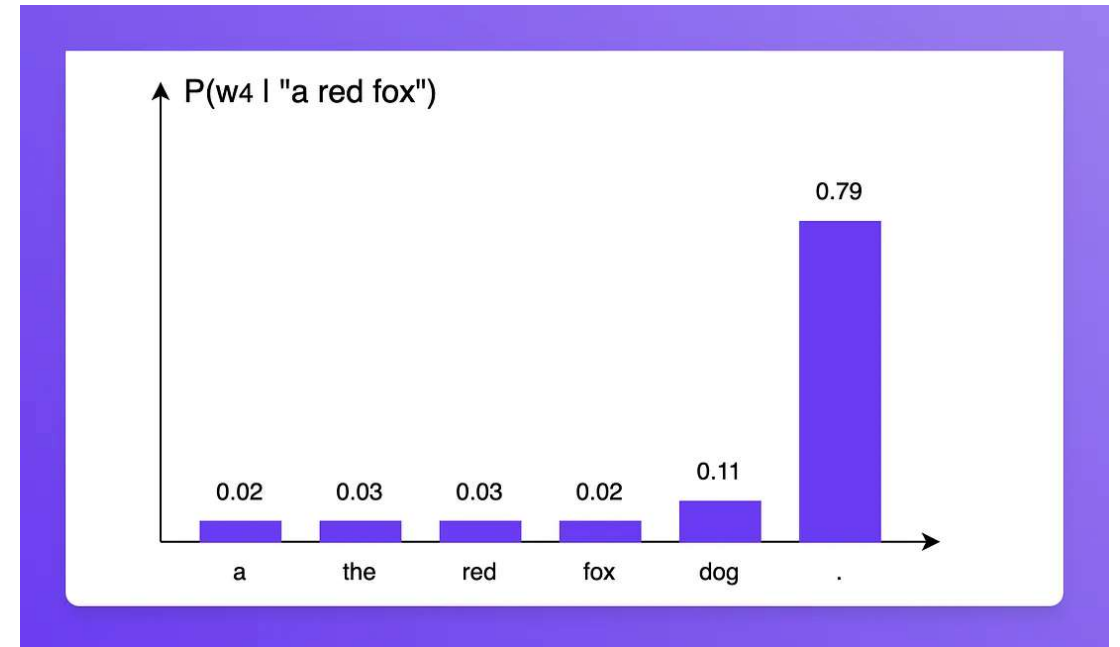
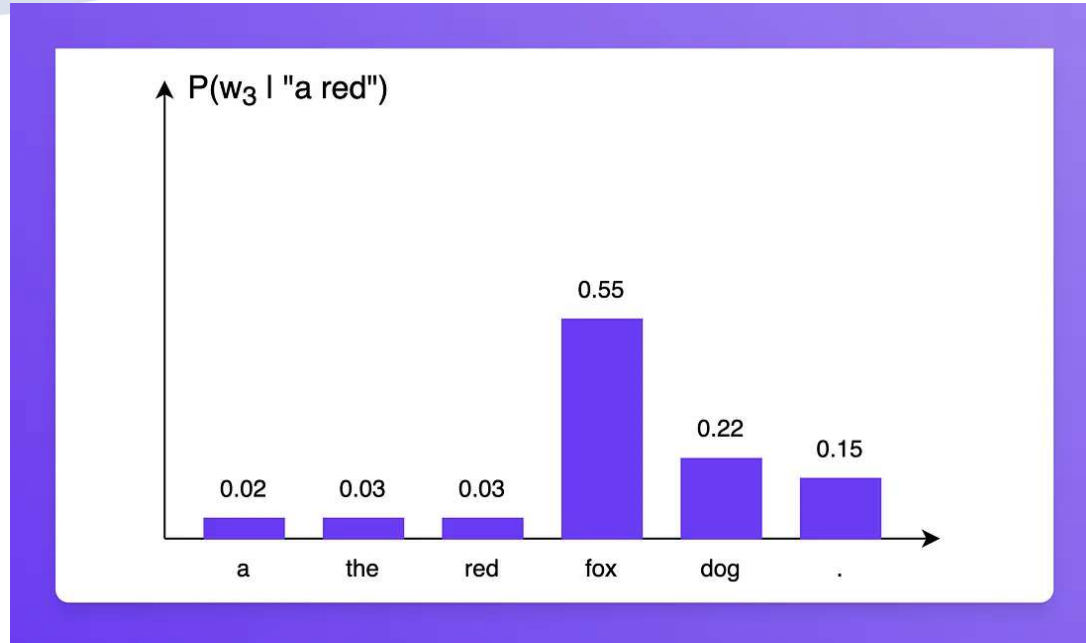
Next, suppose these are the probabilities given by our language model to a generic **second word that follows “a”**:



- The probability of “*red*” as the second word in the sentence after “a” is:

$$P(\text{“red”} | \text{“a”}) = 0.27$$

Similarly, these are the **probabilities** of the **next words**:



Finally, the probability assigned by our language model to the **whole sentence** “a red fox.” is:

$P(\text{“a red fox.”}) =$

$P(\text{“a”}) * P(\text{“red”} | \text{“a”}) * P(\text{“fox”} | \text{“a red”}) * P(\text{“.”} | \text{“a red fox”}) = 0.4 * 0.27 * 0.55 * 0.79 = 0.0469$

$P_{\text{norm}}(\text{“a red fox.”}) = P(\text{“a red fox.”})^{(1 / 4)} = 0.465$

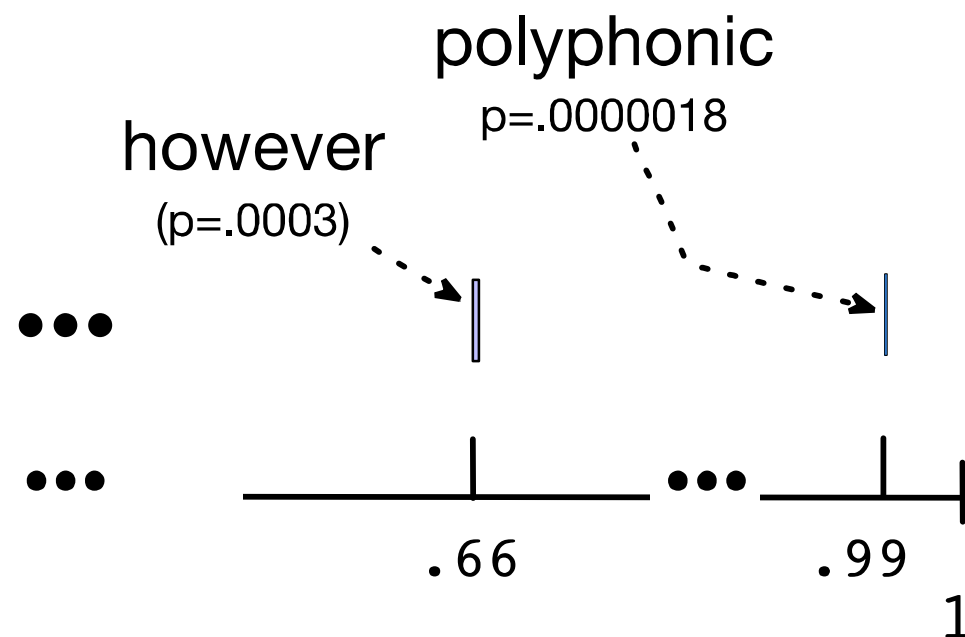
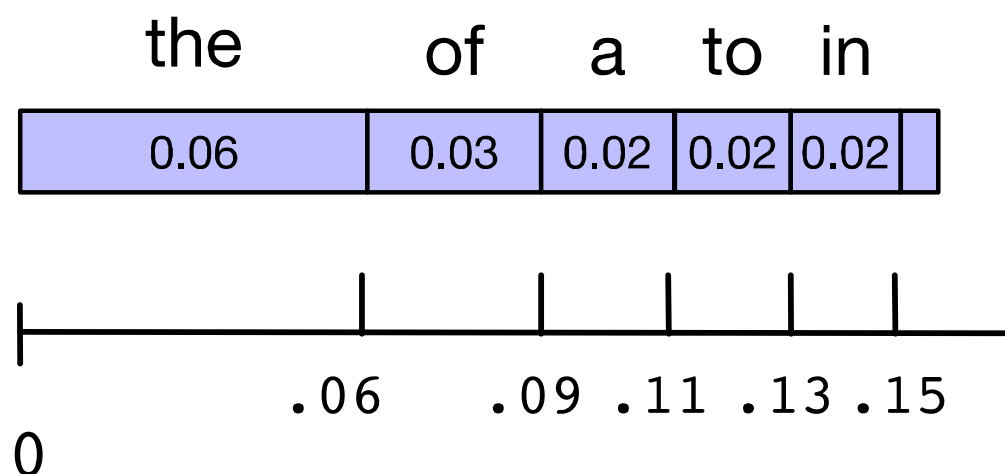
-cont-

- Let's try computing the perplexity with a second language model that assigns equal probability to each word at each prediction.
- Since the language models can predict six words only, the probability of each word will be $1/6$.
- $P(\text{"a red fox."}) = (1/6)^4 = 0.00077$
- $P_{\text{norm}}(\text{"a red fox."}) = P(\text{"a red fox."})^{1/4} = 1/6$
- $PP(\text{"a red fox"}) = 1 / P_{\text{norm}}(\text{"a red fox."}) = 6$
- ...which, as expected, is a higher perplexity than the one produced by the well-trained language model.



Sampling a word from a distribution

- ✓ For the **unigram case**.
- ✓ Imagine all the words of the English language covering the probability space between 0 and 1, each word covering an interval proportional to its frequency.
- ✓ choose a random value between 0 and 1, find that point on the probability line, and print the word whose interval includes this chosen value.
- ✓ suppose we choose a random number, 0.12, look on number line, and that's the word "to", so generate "to" as the next word.
- ✓ We continue choosing random numbers and generating words until we randomly generate the **end-of-sentence token**.



Sampling Bigrams

- We can **extend this idea to n-grams**.
- For example for the **bigram case**:
 - Choose a random bigram ($\langle s \rangle, w$) according to its probability $p(w|\langle s \rangle)$
 - Now choose a random bigram (w, x) according to its probability $p(x|w)$
 - And so on until we choose $\langle /s \rangle$
 - Then string the words together

$\langle s \rangle$ I
I want
want to
to eat
eat Chinese
Chinese food
food $\langle /s \rangle$
I want to eat Chinese food

Shakespeare as corpus

- ✓ $N=884,647$ tokens, $V=29,066$
 - ✓ Shakespeare produced 300,000 bigram types out of $V^2=844$ million possible bigrams.
 - ✓ So 99.96% of the possible bigrams were never seen (have zero entries in the table)
 - ✓ That sparsity is even worse for 4-grams, explaining why our sampling generated actual Shakespeare.
- ✓ If task-specific, use a training corpus that has a similar genre to your task.
 - ✓ If legal or medical, need lots of special-purpose documents
- ✓ Make sure to cover different kinds of dialects and speaker/authors.

The perils of overfitting

- ✓ N-grams only work well for word prediction **if the test corpus looks like the training corpus**
 - But even when we try to pick a good training corpus, the **test set will surprise us!**
 - We need to **train robust models that generalize!**
- ✓ One kind of generalization: **Zeros**
 - Things that **don't ever occur in the training set**
 - But **occur in the test set**



Zeros

For example, consider a **training set** that has the phrases "ate lunch", "ate dinner", "ate a", and "ate the", but happens **not to have had the phrase "ate breakfast"**. The **P(breakfast | ate) in the test set will be 0!**

- Training set:
 - ... ate lunch
 - ... ate dinner
 - ... ate a
 - ... ate the

$$P(\text{"breakfast"} \mid \text{ate}) = 0$$

- Test set
 - ... ate lunch
 - ... ate breakfast

✓ Bigrams with zero probability

- Will hurt our performance for texts where those words appear!
- And mean that we will assign 0 probability to the test set!

✓ And hence we **cannot compute perplexity (can't divide by 0)!**

This sparsity can take many forms

- Verbs have direct objects; those can be sparse too!

Count(w | denied the)

Counts:

3 allegations

2 reports

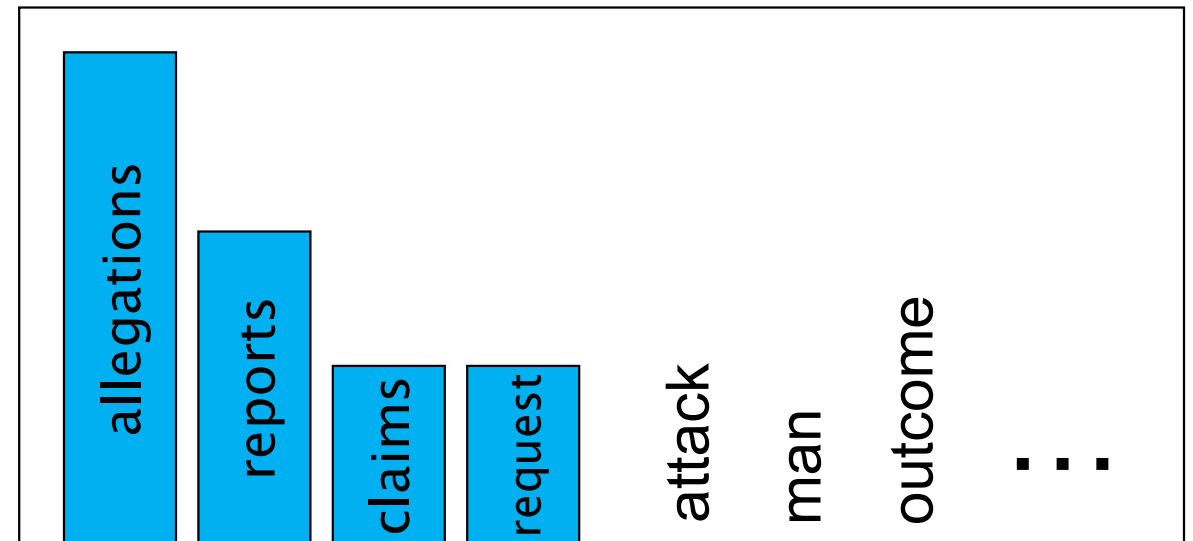
1 claims

1 request

0 attack

0 outcome

7 total



The intuition of smoothing

Smoothing fixes sparsity by taking a little probability mass from seen items and giving some to unseen/rare items.

- When we have sparse statistics:

Count(w | denied the)

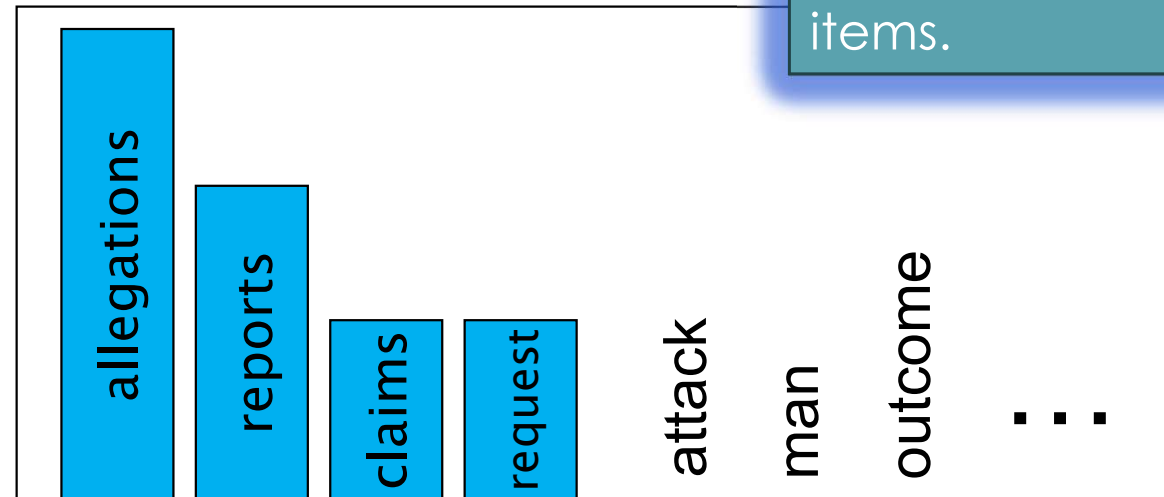
3 allegations

2 reports

1 claims

1 request

7 total



Steal probability mass to generalize better

Count(w | denied the)

2.5 allegations

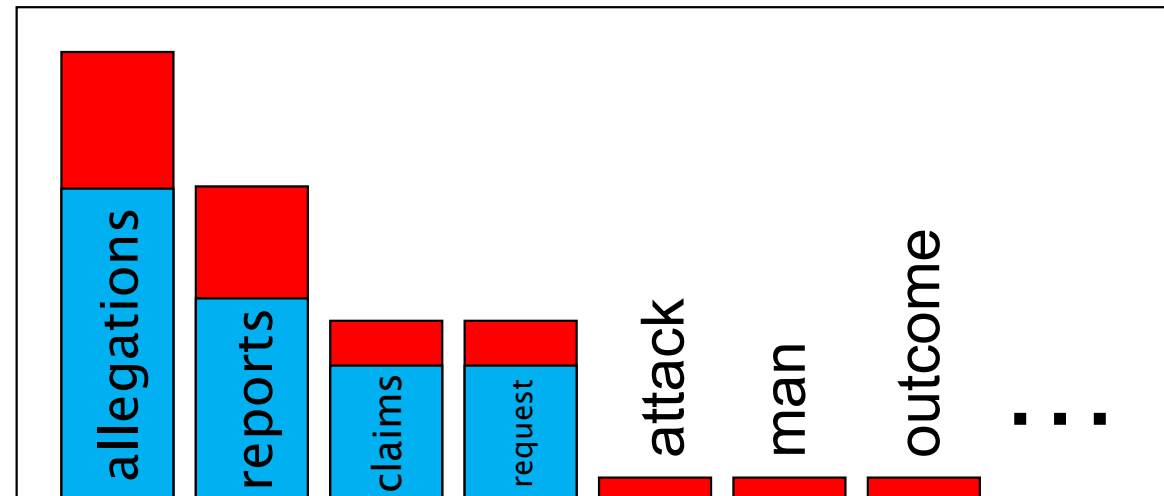
1.5 reports

0.5 claims

0.5 request

2 other

7 total



Add-one estimation

- ✓ Also called **Laplace smoothing**
- ✓ Pretend we saw each word one more time than we did
- ✓ Just add one to all the counts!

✓ **MLE estimate:**

$$P_{\text{MLE}}(w_n | w_{n-1}) = \frac{C(w_{n-1}w_n)}{C(w_{n-1})}$$

✓ **Add-1 estimate:**

$$P_{\text{Laplace}}(w_n | w_{n-1}) = \frac{C(w_{n-1}w_n) + 1}{\sum_w (C(w_{n-1}w) + 1)} = \frac{C(w_{n-1}w_n) + 1}{C(w_{n-1}) + V}$$

Berkeley Restaurant Corpus: Laplace smoothed bigram counts

$$C(w_{n-1}w_n) + 1$$

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Laplace-smoothed bigrams

$$P_{\text{Laplace}}(w_n | w_{n-1}) = \frac{C(w_{n-1}w_n) + 1}{\sum_w (C(w_{n-1}w) + 1)} = \frac{C(w_{n-1}w_n) + 1}{C(w_{n-1}) + V}$$

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

Add-1 estimation is a blunt instrument

- So add-1 isn't used for N-grams:
 - Generally we use interpolation or backoff instead
- But add-1 is used to smooth other NLP models
 - Or we can use **add-k** where $0 < k < 1$
 - It's used in naive bayes text classification
 - In domains where the number of zeros isn't so huge.

Backoff and Interpolation

- Sometimes it helps to use a simpler model
 - Condition on **less** context for contexts you know **less** about
 - **Backoff:**
 - If **enough evidence**, use **trigram** $P(w_n|w_{n-2}w_{n-1})$
 - If **not**, use **bigram** $P(w_n|w_{n-1})$
 - Else **unigram** $P(w_n)$
 - **Interpolation:**
 - mix unigram, bigram, trigram
- ✓ Interpolation often works better

Linear Interpolation

- ✓ Combine different order n-grams by linearly interpolating them.
- ✓ Estimate the trigram probability $P(w_n|w_{n-2}w_{n-1})$ by mixing together the unigram, bigram, and trigram probabilities, each weighted by a λ :

- ✓ Simple interpolation

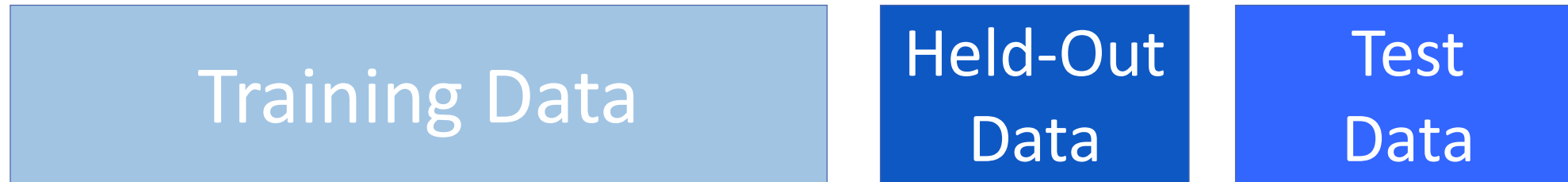
$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) = & \lambda_1 P(w_n|w_{n-2}w_{n-1}) \\ & + \lambda_2 P(w_n|w_{n-1}) \\ & + \lambda_3 P(w_n) \quad \sum_i \lambda_i = 1\end{aligned}$$

- ✓ In a slightly more sophisticated version of linear interpolation, each λ weight is computed by conditioning on the context:

$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) = & \lambda_1(w_{n-2}^{n-1}) P(w_n|w_{n-2}w_{n-1}) \\ & + \lambda_2(w_{n-2}^{n-1}) P(w_n|w_{n-1}) \\ & + \lambda_3(w_{n-2}^{n-1}) P(w_n)\end{aligned}$$

How to set λ s for interpolation?

- Use a **held-out** corpus



- Choose λ s to maximize probability of held-out data:
 - Fix the N-gram probabilities (on the training data)
 - Then search for λ s that give largest probability to held-out set

Backoff

Suppose you want:

$P(\text{pancakes} | \text{delicious soufflé})$

- If the trigram probability is 0, use the bigram

• $P(\text{pancakes} | \text{soufflé})$

- If the bigram probability is 0, use the unigram

• $P(\text{pancakes})$

Complication: need to discount the higher-order ngram so probabilities don't sum higher than 1

- Backoff without discounting (not a true probability)

$$S(w_i | w_{i-k+1}^{i-1}) = \begin{cases} \frac{\text{count}(w_{i-k+1}^i)}{\text{count}(w_{i-k+1}^{i-1})} & \text{if } \text{count}(w_{i-k+1}^i) > 0 \\ 0.4S(w_i | w_{i-k+2}^{i-1}) & \text{otherwise} \end{cases}$$

$$S(w_i) = \frac{\text{count}(w_i)}{N}$$

Summary: What to do if you never saw an n -gram in training

Smoothing: Pretend you saw every n -gram one (or k) times more than you did

- A blunt instrument (replacing a lot of zeros) but sometimes useful

Backoff: If you haven't seen the trigram, use the (weighted) bigram probability instead

- Weighting is messy; "stupid" backoff works fine at web-scale

Interpolation: (weighted) mix of trigram, bigram, unigram

- Usually the best! We also use interpolation to combine multiple LLMs